

AtlasGuard

Linux-native ML observability + explainable runtime security

Candidate engineering portfolio tailored to Canonical, SurveyMonkey and Sysdig

Leonardo Rosati - BSc Computer Science, University of Bologna (expected 2027)

Python • C/C++ • Java • Linux • Docker • FastAPI • ML evaluation • security tooling

6 tests passing • Python 3.11/3.12 CI • Docker build • PSI drift demo • explainable runtime rules

What I built

Observe - Linux host telemetry + inference telemetry enter a typed FastAPI contract.

Detect - PSI flags model-feature distribution shift; deterministic rules flag runtime anomalies.

Operate - Prometheus metrics, bounded state, failure-tolerant agent loop and reproducible demos.

Validation: 6 tests passing • PSI 7.83 after shift • 5 runtime findings • 2 Python versions in CI

Architecture

Linux agent -> typed telemetry contract -> FastAPI control plane

Inference service -> latency / feature signal -> drift detector

Runtime events -> explainable rule engine

Control plane -> bounded event store + Prometheus metrics

Engineering principles

Bounded - bounded dequeues instead of unbounded in-memory accumulation.

Resilient - network failures do not terminate collection.

Least privilege - non-root container, no added capabilities, read-only filesystem.

Explainable - stable rule IDs + human reasons; PSI is reproducible math.

Privacy-aware - metadata / numeric features, not raw prompts or user text.

Honest - eBPF, durable storage and distributed scale are roadmap items, not claimed features.

Canonical fit

Target: Graduate Software Engineer - Open Source & Linux

Linux agent + systemd unit + container hardening

Typed Python modules, bounded state and failure-tolerant sender

CLI, quick-start, threat model and reproducible CI

Discussion: Ubuntu packaging, journald, profiling under load and upstream-friendly design.

Canonical demo

24 seconds - Linux, validation, resilience and least privilege

Standalone video: [videos/AtlasGuard_Canonical_Demo.mp4](#)

SurveyMonkey fit

Target: Machine Learning Engineer - Italy / Remote

Model/service telemetry contract decoupled from model implementation

PSI baseline + rolling window surfaces statistical distribution shift

Prometheus latency + drift + system health metrics

Discussion: a model can degrade while the API remains healthy, so monitoring should be model-agnostic.

Actual ML drift demonstration

Reference window vs shifted production window

PSI = 7.83

Threshold = 0.20

Result: DRIFTED = TRUE

The API remains healthy; the model-reliability signal changes.

SurveyMonkey demo

24 seconds - telemetry, PSI and Prometheus signal

Standalone video: [videos/AtlasGuard_SurveyMonkey_Demo.mp4](#)

Sysdig fit

Target: Junior AI & Product Enablement Specialist - Italy / Remote

Docker/Python lab with repeatable normal + suspicious scenarios

Architecture + threat model + explicit non-goals

Linux/container runtime events with explainable findings

Discussion: build the lab, explain each finding, then swap synthetic events for Falco/eBPF collection.

Sysdig-style hands-on lab

1 Start - control plane + metrics

2 Observe - normal telemetry

3 Trigger - root shell / port 4444

4 Explain - rule -> fields -> finding

5 Extend - swap in Falco/eBPF source

Sysdig demo

24 seconds - runtime event -> explainable finding

Standalone video: [videos/AtlasGuard_Sysdig_Demo.mp4](#)

What AtlasGuard does not claim

Not eBPF yet - runtime events are currently user-space inputs.

Not distributed storage - bounded in-memory state is deliberate for a reviewable prototype.

Not cloud-scale ML - this demonstrates platform mechanics, not AWS/Kafka/EKS/SageMaker production tenure.

The code tells the same story as the CV; roadmap items stay labeled as roadmap.

What I would build next

Canonical: Debian/Ubuntu packaging, journald, profiling, upstream workflow, Ubuntu integration tests

SurveyMonkey: per-feature baselines, online evaluation, durable storage, OpenTelemetry, Kubernetes, SLO/load tests

Sysdig: Falco adapter, eBPF source, Kubernetes lab, workshop courseware, false-positive tuning exercise

Why I am showing you this

I am early-career, so I want the work to be inspectable.

AtlasGuard - ML observability + runtime security control plane

polyglot-alpha-beta-arena - algorithms + C++ autodiff + Java DQN + multi-language alpha-beta

Private R&D - HypoWeb scientific reasoning • IncidentLens defensive security analysis

The goal is not to look senior on paper. The goal is to show evidence of how I think as an engineer.

Links, demos and sources

GitHub: github.com/IEoNardoToP/IEoNardoToP/tree/main/projects/atlasguard

LinkedIn: [linkedin.com/in/leonardo-rosati-97b863273/](https://www.linkedin.com/in/leonardo-rosati-97b863273/)

Email: leonardo.rosati@icloud.com

Prepared 6 October 2026